Tuning Your Privacy on Outsourced Data Through Semi-Trusted Helpers

Qingqing Xie, Yantian Hou, *Member, IEEE*, Fatong Zhu, Ke Cheng, and Hanyu Quan, *Senior Member, IEEE*

***Abstract*—Tuning the data precision before outsourcing is a common method to protect privacy against untrusted third parties. Usually, the data owner needs to add noises into the data and share each noisy version with certain users. It is a risky task, especially for the non-technical data owner, such as young children. To solve this problem, a data owner could enlist a helper to enact the noise-tuning policy. However, in practice, the helper is often only semi-trusted by the data owner. For example, consider parents applying parental controls to their child's devices. The parents are trusted for their technical expertise, yet they may also be curious about the child's privacy. Hence, how to enforce the delegation control on data precision while maintaining data privacy remains an underestimated challenge. To address this issue, we propose a helper-assisted data precision tuning model, then apply our generic crypto primitive—fine-grained access precision control (FAPC) to let a semi-trusted helper tune the data precision, while the data owner maintains complete control over the original data privacy. We theoretically and experimentally prove that our scheme achieves strong privacy guarantee, control flexibility, and policy enforcement simultaneously in a delegated data privacy control scenario.**

***Index Terms*—Access control delegate, attribute-based encryption, secure computing, privacy preservation, parental control.**

I. Introduction

TUNING data precision before outsourcing is a common privacy-preserving technique. For instance, users add noise to their real-time GPS data before sharing it with cloud-based location services. Noise degrades precision, trading utility for security. This general approach has been widely utilized in various cloud-computing data security solutions, such as differential privacy [2], [3], [4].

However, tuning data precision is a challenging task, particularly for non-technical data owners (DO) like children or the elderly. Many people lack the security expertise to make informed decisions. For example, teenagers posting photos on social media may understand that different friends deserve different levels of location privacy, but often cannot implement such differentiation.

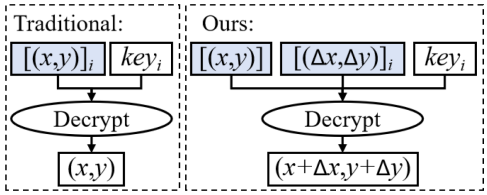


Fig. 1. The traditional fine-grained access control (left), and our fine-grained access precision control (right) combined with homomorhpic operation of a shared component ($[(x,y)]$) and an identity-associated component ($[(\Delta x,\Delta y)]_i$). $[(x,y)]$ denotes the encryption of raw location (x,y), $[(\Delta x,\Delta y)]_i$ denotes the encryption of the noise $(\Delta x,\Delta y)$ under the identity i, key_i denotes the decryption key of the identity i.

Handling such intricate tasks requires expert help, ideally from a parent, not the child. Delegation becomes the natural solution. A child entrusts their phone to a parent to fine-tune location precision and manage sharing with third parties. Existing parental control solutions [5], [6], [7] are neither secure nor enforceable. Handing over the device grants the parent full access to private data and control, which may cause discomfort. Moreover, the child can easily disable the controls after reclaiming the device. To our knowledge, no existing work achieves both enforceable delegation of precision control and data privacy.

In this study, we empower DO to enlist semi-trusted helpers to implement data noising policy. The DO only manages data throttling, significantly reducing their burden. The helper is semi-trusted. The DO relies on the helper's expertise but does not share raw data. This model fits parent-child scenarios well. First, limiting the DO's involvement to throttling the raw data protects their privacy even from the helper. Second, helpers are often more vigilant about privacy threats, making them better suited to determine the noising policy. No prior work has achieved this functionality.

To realize this goal, we combine fine-grained access control with homomorphism. We add different noise levels to the same raw data and share the noisy versions with different users. For instance, via a semi-trusted helper, a DO distributes ciphertexts of zero, light, and heavy noise to family, friends, and a "finding-nearest-restaurant" application, respectively. The DO publishes the encrypted real-time location coordinates (x,y). Each user can only decrypt their authorized noisy version through online homomorphic operations, i.e., adding an identity-associated noise component to a dynamically shared location component,

Received 31 December 2023; revised 21 May 2026; accepted 25 June 2026. Date of publication 26 June 2026; date of current version 1 September 2026. *(Corresponding author: Qingqing Xie.)*

Qingqing Xie and Fatong Zhu are with the School of Computer Science and Communication Engineering, Jiangsu University, Zhenjiang 212013, China (e-mail: xieqq@ujs.edu.cn; ftzhu@stmail.ujs.edu.cn).

Yantian Hou is with Palo Alto Networks, Inc., Santa Clara, CA 95054 USA (e-mail: yhou@paloaltonetworks.com).

Ke Cheng is with the School of Computer Science and Technology, Xidian University, Xi'an 710126, China (e-mail: kechengstu@gmail.com).

Hanyu Quan is with the College of Computer Science and Technology, Huaqiao University, Xiamen 361021, China (e-mail: quanhanyu@hqu.edu.cn).

Digital Object Identifier 10.1109/TDSC.2026.3707948

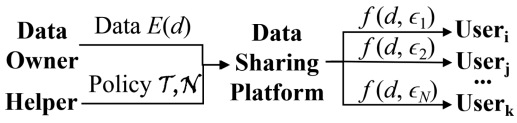


Fig. 2. Our system model: The DO uploads encrypted raw data $E(d)$ to the data sharing platform. The helper enforces policies represented by a set of access trees $\mathcal{T}$ and a set of noises $\mathcal{N}$. Each user fetches the data and decrypts to obtain their authorized noisy version $f(d, \epsilon_i)$.

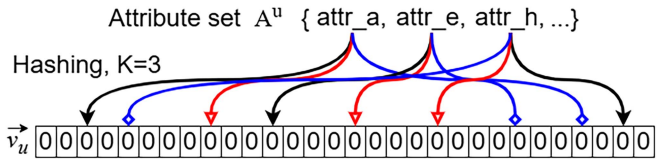


Fig. 3. Bloom filter vector generation example: 3 hash functions, same color and arrow-end shape = same function.

as illustrated in Fig. 1. Our approach stems from the necessity for fine-grained access control over the same data at varying precisions for different users.

Our design aims to meet three requirements: 1) Flexibility. Users receive precision levels according to a helper-enacted fine-grained policy. 2) Security. Users access only their designated version based on attributes. The helper cannot see the raw data or noisy data. 3) Lightweight. For the non-professional DO, minimal computation is required, even when dealing with dynamic data.

To simultaneously fulfill these requirements, we propose a cryptographic primitive with a novel decoupling technique. The shared (raw/precise) data and identity-associated noises are encrypted separately while preserving additive homomorphism. To decrypt, each user first "plugs" their noise ciphertext into the shared ciphertext, and unlocks the composite with the secret key to derive the noisy value.

Decoupling is highly practical for the DO. They need only encrypt the shared data component at runtime, yielding constant overhead regardless of granularity (number of noisy levels). Fine-grained policy enforcement and noise encryption can be fully offloaded to the helper (e.g., a parent). This relieves non-professional DO lacking access control expertise.

Recent cryptographic works combine homomorphic encryption with attribute-based encryption (ABE) [8], [9], [10], [11], [12], [13], [14], [15] to achieve fine-grained access over arithmetic results. Compared to our work, they focus on homomorphic operations across ciphertexts that are encrypted under the same or different attributes. Many of these schemes suffer from significant ciphertext size inflation, imposing heavy encryption and storage overhead on DO. Moreover, they require the DO to enact the noising policy, which are unsuitable for non-technical DO.

Our preliminary work [1] theoretically validated the correctness and security of our cryptographic algorithm. Here, we further explore its practical use in secure computing. We establish an identity-differentiated data sharing protocol and apply it to location obfuscation, which is a widely studied privacy mechanism for location-based services (LBS) [16], [17], [18]. In our protocol, the helper pre-loads the policy and noise ciphertext to all users via a data sharing platform. At runtime, the DO encrypts the raw data component, and distributes it through the platform. Each user downloads the encrypted raw data and decrypts after embedding the corresponding noise ciphertext. By our design, each user could only access the data at the precision enforced by the helper.

In this paper, the main contributions are:

1) We propose an owner-helper data precision tuning model, and apply our generic crypto primitive-fine-grained access precision control (FAPC) to let a semi-trusted helper tune precision without leaking raw data.
2) Using our FAPC,[1] we design an identity-differentiated location obfuscation (IDLO) protocol that is lightweight for DO and precisely controls the location privacy leakage to different users.
3) We formally prove the security of our scheme using proof-by-reduction and game-based methods.
4) We establish a prototype on a commercial cloud platform, where one DO performs the 1-to-n data sharing towards 32 users at different precision levels. Extensive experiments validate its efficacy and complexity.

Section II reviews the preliminaries. Section III describes the system model and formally defines the problem. Section IV presents the proposed FAPC primitive. Section V details the design of the IDLO protocol. Section VI analyzes the complexity of our scheme. Section VII illustrates the implementation of our cloud-based application, and compares its performance with two state-of-the-art works. Section VIII discusses our scheme in terms of complexity, security and application. Section IX reviews related work. Section X concludes the paper.

II. Preliminaries

We first briefly review bilinear map and the ciphertext-policy ABE (CP-ABE) scheme.

A. Bilinear Map

Let $\mathbb{G}_0$ and $\mathbb{G}_T$ be two multiplicative cyclic groups of prime order q, and g be a generator of $\mathbb{G}_0$. The bilinear map e is defined as: e: $\mathbb{G}_0 \times \mathbb{G}_0 \rightarrow \mathbb{G}_T$, which has the following four properties:

- *Bilinearity:* $\forall u, v \in \mathbb{G}_0$,$\forall a, b \in \mathbb{Z}_q$,$e(u^a, v^b) = e(u, v)^{ab}$.
- *Symmetry:* $\forall u, v \in \mathbb{G}_0$, $e(u, v) = e(v, u)$.
- *Non-degeneracy:* $e(g, g) \neq 1$.
- *Computability:* $\forall u, v \in \mathbb{G}_0$, there is an efficient algorithm to compute $e(u, v)$.

B. CP-ABE

The CP-ABE scheme was proposed to achieve fine-grained access control [19]. In CP-ABE, a user's private attribute key is associated with an attribute set. The ciphertext is associated with an access policy.[2] A user will succeed in decrypting a ciphertext if and only if the user's attribute set satisfies the access policy. The CP-ABE scheme consists of four algorithms:

- $Setup(1^\lambda) \rightarrow mk, pk$: It takes the security parameter 1^λ, and outputs a primary key mk and a public key pk.

[1]The main source code is publicly available at https://github.com/xieqqjava/FAPC_AsiaCCS.

[2]We refer the reader to [19] for details.

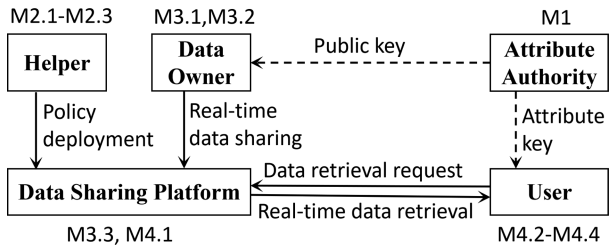


Fig. 4. The IDLO protocol workflow: It is divided into initialization (M1, M2.1–M2.3), online data publishing (M3.1–M3.3), and data retrieval (M4.1–M4.4).

- $Encrypt(pk, m, T) \rightarrow \mathcal{E}_T(m)$: It takes as input pk, a message m, and an access tree T over the universe of attributes, then encrypts m as a ciphertext $\mathcal{E}_T(m)$.
- $KeyGenerate(pk, mk, \mathbb{A}^u) \rightarrow \mathbb{SK}_u$: It takes pk, mk, and a user u's attribute set $\mathbb{A}^u$ as input, then outputs u's secret attribute key $\mathbb{SK}_u$.
- $Decrypt(pk, \mathcal{E}_T(m), \mathbb{SK}_u) \rightarrow m$: It takes pk, $\mathcal{E}_T(m)$, and $\mathbb{SK}_u$ as input, then outputs the message m if the user u's attribute set satisfies the access tree associated with $\mathcal{E}_T(m)$.

III. Problem Statement

A. Problem Definition

The problem we are studying is fine-grained access precision control. Our goal is to enable the DO, with the assistance of a semi-trusted helper, to add different levels of noise to shared data, and disclose each noisy version only to authorized users. Unlike existing state-of-the-art works, our solution aims to be lightweight for the DO, and thereby scalable to support many users. Meanwhile, it should be privacy-preserving, thus no unauthorized entity (including semi-trusted helper, unauthorized users, untrusted cloud platform) could access the raw and noisy data.

For example, in a location-sharing scenario, a parent (helper) can add various levels of noise to a child's (DO's) real-time location information before sharing it with different types of friends or third-party service providers (users). Throughout this process, the parent performs the primary tasks, including adding noise and managing access control, but cannot see the child's true location. The child only runs minimum calculations to safeguard the true location.

Before formulating our problem, we first define the noise function and the access precision control policy as follows:

Definition III.1 (Noisy Data): Use $\mathbb{DS}$ to denote the data space. Given a data $d \in \mathbb{DS}$, a noise $\epsilon \in \mathbb{DS}$, and any noise function f, a noisy data is denoted as $d_\epsilon = f(d, \epsilon)$.

Definition III.2 (Access Precision Control Policy): Use $\mathbb{US}$ to denote the user space. The access precision control policy set Γ is a set of identity-precision pairs (u, ϵ_i), each associating a user's identity $u \in \mathbb{US}$ with a noisy version ϵ_i.

The noise function f could be in any form. In this work, we consider the basic additive function. Given the definition of noisy data and the access precision control policy, we could then formulate the FAPC problem as follows:

Definition III.3 (FAPC Problem): Given any data $d \in \mathbb{DS}$ and a policy set Γ, find a scheme Π, such that the noisy data $d_{\epsilon_i} = f(d, \epsilon_i)$ could be revealed to user u, iff $(u, \epsilon_i) \in \Gamma$.

Briefly, the FAPC problem is to find a scheme Π such that a user u could only access its permitted noisy data $f(d, \epsilon_i)$ according to a pre-defined policy Γ. In practice, the policy Γ is normally defined by the helper, which associates each noisy version with one or more users.

B. A Naive Solution

Very few existing solutions consider introducing a semi-trusted helper to assist a non-expert DO, and thus they fail to address the FAPC problem in parental-control-type scenarios. A straightforward yet naive solution is to directly apply popular fine-grained access control mechanisms, such as lightweight CP-ABE [20] or UR-CP-ABE [21], where unfortunately the DO simultaneously acts as the helper to formulate access policies and add noise.

In such approaches, the DO first determines the access policy and adds noise ϵ_i to the raw data d. Assuming there are N precision levels in total, the noisy data can be represented as a vector $\vec{\mathbf{d}}_\epsilon = [d_{\epsilon_1}, d_{\epsilon_2}, \ldots d_{\epsilon_N}]$, containing all N noisy versions of the raw data d. To ensure security against semi-honest cloud and unauthorized users, the DO encrypts each noisy version using CP-ABE schemes [20], [21], i.e., $Enc(\vec{\mathbf{d}}_\epsilon) = [\mathcal{E}_{T_1}(d_{\epsilon_1}), \mathcal{E}_{T_2}(d_{\epsilon_2}), \ldots, \mathcal{E}_{T_N}(d_{\epsilon_N})]$. The resulting encrypted vector $Enc(\vec{\mathbf{d}}_\epsilon)$ is uploaded to the cloud platform where users can retrieve it upon requests. Each user can only access the noisy version authorized by the DO.

C. Complexity Issue

The practical limitation of such a solution lines in its complexity. To enforce a given policy Γ, the DO must generate each noisy data d_{ϵ_i} and encrypt them using CP-ABE, which is expensive that is often unfulfillable for a non-technical DO. This workload becomes even more non-negligible when the policy size is large [22], or when the raw data d is generated or updated frequently (e.g., a real-time stream of location coordinates in LBS).

Combining homomorphic encryptions with ABE [8], [9], [10], [11], [12], [13], [14], [15] enables online computations, however is non-trivial and often yields to significantly large ciphertexts, increasing both the encryption cost for the DO and the storage overhead on the platform. Moreover, existing schemes mainly focus on performing computations across different users' attributes. Directly adapting them to our setting would fail to meet our security requirement, because they are not designed to reveal only the final computational results to authorized users. Instead, they may expose intermediate values or produce results accessible to parties beyond the intended recipient.

Our goal is to solve the privacy tuning problem on outsourced data through a semi-trusted helper. The desired solution must be: 1) Secure against unauthorized users, untrustworthy cloud platforms and the semi-trusted helper; 2) Flexible for the helper to control data precision; and most importantly, 3) Lightweight to operate in practice.

TABLE I
MAJOR NOTATIONS

$m = e(g,g)^d$	shared component in $\mathbb{G}_T$ transformed from raw data $d \in \mathbb{Z}_q$
$\tau_i = e(g,g)^{\epsilon_i}$	i-th piece of identity-associated (noise) component transformed from original noise $\epsilon_i \in \mathbb{Z}_q$
$\mathcal{N} = \{\tau_i\}_{i\in[1,N]}$	set of noises
$d_{\epsilon_i} = f(d,\epsilon_i)$	data d's i-th noisy version before transformation, where $f : \mathbb{Z}_q \times \mathbb{Z}_q \to \mathbb{Z}_q$ is a noise function on $\mathbb{Z}_q$
$m_{\tau_i} = f'(m,\tau_i)$	data m's i-th noisy version after transformation, where $f' : \mathbb{G}_T \times \mathbb{G}_T \to \mathbb{G}_T$ is a noise function on $\mathbb{G}_T$
$\mathcal{T} = \{T_i\}_{i\in[1,N]}$	set of access trees, where T_i determines who can access the noisy data $f'(m,\tau_i)$
$\mathbb{A}^u$	user u's attribute set
$\mathbb{SK}_u$	user u's secret attribute key

IV. OUR CRYPTOGRAPHIC PRIMITIVE – FAPC

In this section, we present our cryptographic primitive FAPC, that is partially based on the CP-ABE scheme [19]. FAPC decouples the ciphertext into a dynamically shared component and multiple identity-associated components. At decryption, the shared component is reused and paired with the identity-associated components. Table I summarizes the major notations. The detailed scheme is presented in our previous work [1], and consists of the following six algorithms:

- $Setup(1^\lambda) \to mk, pk$: A probabilistic algorithm that takes a security parameter 1^λ, and outputs a primary key mk and a public key pk.
- $Encrypt'(pk, m, k, T_\forall) \to CT_m$: A probabilistic algorithm that takes the public key pk, a plaintext m (the shared component), a session key k and an access tree $T_\forall$. It outputs the ciphertext CT_m of data m. Here $T_\forall$ is defined such that a user's attribute set satisfies $T_\forall$ if it satisfies at least one of the individual trees $T_i (i \in [1, N])$.
- $Encrypt''(pk, \mathcal{N}, \mathcal{T}, k) \to \mathcal{E}_{T_\forall}(k), CT_\mathcal{N}$: A probabilistic algorithm that takes the public key pk, a plaintext noise set $\mathcal{N} = \{\tau_i\}_{i\in[1,N]}$ (the identity-associated components), a set of access trees $\mathcal{T} = \{T_i\}_{i\in[1,N]}$ and a session key k. It outputs the ciphertext $\mathcal{E}_{T_\forall}(k)$ of k and the ciphertext set $CT_\mathcal{N} = \{\mathcal{E}_{T_i}(A_{\tau_i})\}_{i\in[1,N]}$ of $\mathcal{N}$. Note that A_{τ_i} is obtained by randomizing the noise τ_i and is simply called a noise-hidden component.
- $Update(pk, m, CT_m, k, m') \to CT_{m'}$: A probabilistic algorithm that generates a ciphertext $CT_{m'}$ for the new raw data m', in two steps:
  Step 1: Compute the ciphertext increment $CT_{m\to m'}$ using the public key pk, the session key k, the new raw data m', the old raw data m and its ciphertext CT_m;
  Step 2: Generate the new ciphertext $CT_{m'}$ from $CT_{m\to m'}$ and CT_m.
- $KeyGenerate(pk, mk, \mathbb{A}^u) \to \mathbb{SK}_u$: A probabilistic algorithm that takes the public key pk, the primary key mk, and an attribute set $\mathbb{A}^u$ of a user u. It outputs a secret attribute key $\mathbb{SK}_u$ associated with $\mathbb{A}^u$.
- $Decrypt(pk, \mathcal{E}_{T_\forall}(k), \mathcal{E}_{T_i}(A_{\tau_i}), CT_m, \mathbb{SK}_u) \to f'(m,\tau_i)$ or null: A deterministic algorithm that uses the attribute key $\mathbb{SK}_u$ to recover the noisy data $f'(m,\tau_i)$, if $\mathbb{A}^u$ satisfies T_i. It proceeds in three steps:
  Step 1: Decrypt $\mathcal{E}_{T_\forall}(k)$ to obtain the session key k;
  Step 2: Decrypt the noise ciphertext $\mathcal{E}_{T_i}(A_{\tau_i})$ to obtain the noise-hidden component A_{τ_i};
  Step 3: Compute $f'(m,\tau_i)$ from CT_m, k and A_{τ_i}.

The correctness and security of our FAPC are defined below.

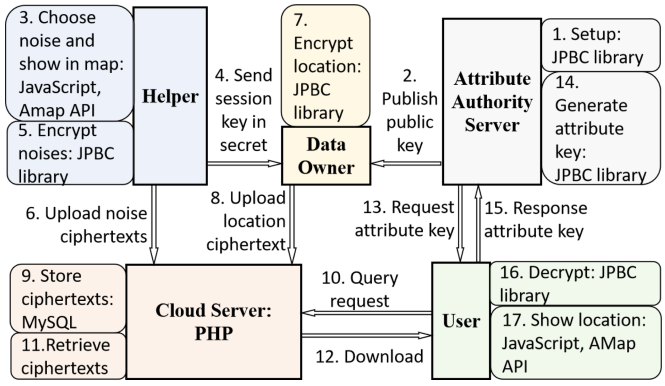


Fig. 5. The architecture of our IDLO system.

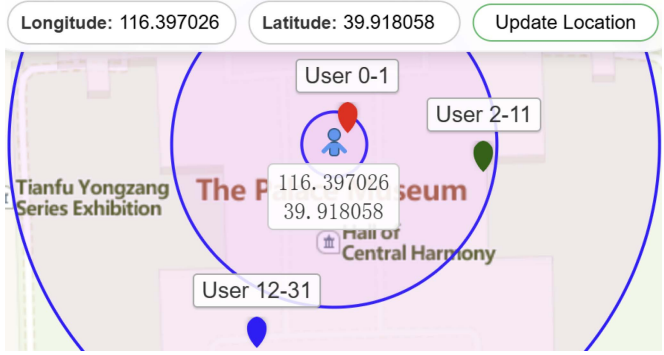


Fig. 6. DO-side console (partially displayed due to limited page space).

(a) User 2 (b) User 31

Fig. 7. User-side consoles at User 2 and User 31.

A. Correctness Definition

The correctness of the FAPC scheme is that the user is able to access the noisy data if and only if the user's attribute set satisfies the corresponding access tree. Formally:

Definition IV.1 (FAPC Scheme Correctness): For algorithms $(Encrypt', Encrypt'', Decrypt)$, we have $Decrypt_{sk_i}(Encrypt'(m), Encrypt''(\tau_i)) = f'(m,\tau_i)$, iff $\mathbb{A}^u$ satisfies the corresponding tree T_i. Otherwise, the $Decrypt$ algorithm outputs $null$.

TABLE II
COMPARISON OF COMPUTATIONAL COMPLEXITIES

Scheme	Setup	KeyGenerate	Encrypt at DO	Encrypt at Helper	Update	Decrypt
Lightweight CP-ABE [20]	$O(\|AS\|)$	$O(\|\mathbb{A}^u\|)$	$O(N \cdot X)$	–	$O(N \cdot X)$	$O(X)$
UR-CP-ABE [21]	$O(\|AS\| + \|NU\|)$	$O(\|\mathbb{A}^u\| + \log\|NU\|)$	$O(N \cdot X)$	–	$O(N \cdot X)$	$O(X)$
Ours	$\boldsymbol{O(1)}$	$O(\|\mathbb{A}^u\|)$	$\boldsymbol{O(1)}$	$O(N \cdot X)$	$\boldsymbol{O(1)}$	First time: $O(X)$ Subsequent: $\boldsymbol{O(1)}$

N: the number of noise levels. X: the number of leaves per access tree. $|\mathbb{A}^u|$: the number of user attributes. $|AS|$: the size of attribute space. $|NU|$: the maximum number of users. –: not applicable or not supported.

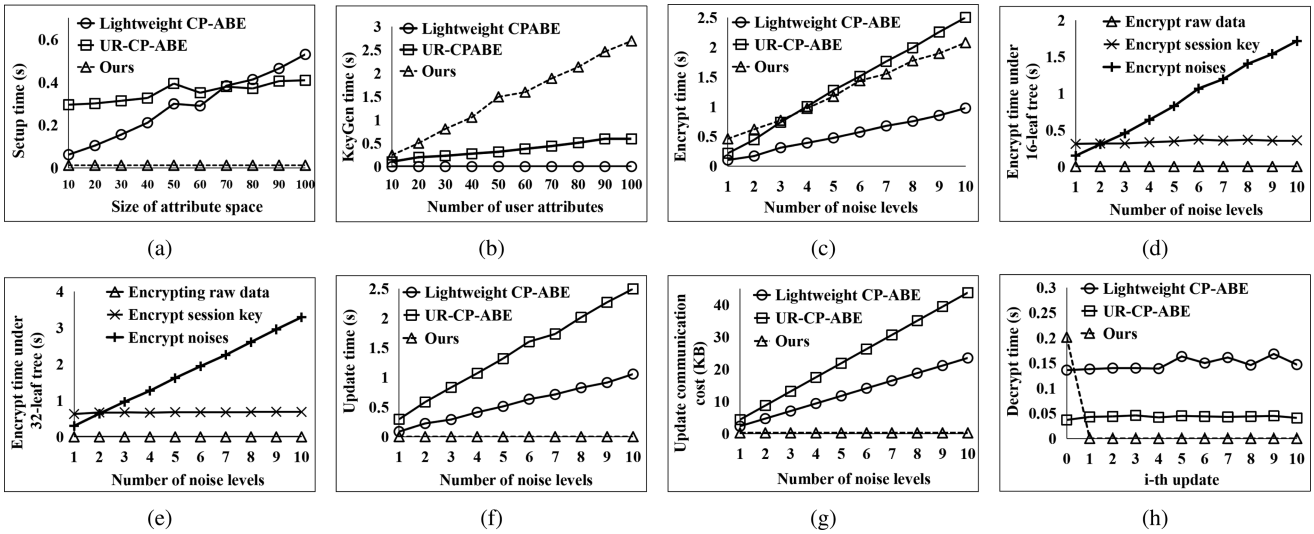


Fig. 8. Performance evaluation: (a) Setup time comparison; (b) Key generation time comparison; (c) Encryption time comparison under 16-leaf tree; (d)–(e) Encryption step-wise time cost under 16-leaf and 32-leaf access trees; (f) Update time comparison; (g) Update communication cost comparison; (h) Decryption time comparison.

B. Security Definition

Our scheme should be secure in the following security model. Briefly, the adversary in the following game tries to distinguish an arbitrary but equal-length $\{m, \{\tau\}\}$ pair, with arbitrary size of the set $\{\tau\}$. Using Π to denote our FAPC scheme, the security definition is stated via the following experiment $\boldsymbol{Exp}_{\mathcal{A},\Pi}^{\mathcal{T}-cpa}$, which is a variation of indistinguishability under chosen plaintext attack (CPA):

Setup. The challenger runs the $Setup$ algorithm and gives the public key pk to the adversary $\mathcal{A}$.

Phase 1. The adversary $\mathcal{A}$ repeatedly makes attribute key queries corresponding to attributes $\mathbb{A}^1$, $\mathbb{A}^2$, …, $\mathbb{A}^{n_1}$. The challenger responds by running the attribute key generation algorithm to generate the corresponding attribute key $\mathbb{SK}_j$, for each $j \in [1, n_1]$.

Challenge. The adversary $\mathcal{A}$ submits two sets of plaintexts $\{m_0, \mathcal{N}_0 = \{\tau_i^0\}_{i\in[1,N]}\}$ and $\{m_1, \mathcal{N}_1 = \{\tau_i^1\}_{i\in[1,N]}\}$, where all counterparts are of equal length. In addition, the adversary $\mathcal{A}$ gives a challenge set of access trees $\mathcal{T}^* = \{T_i\}_{i\in[1,N]}$ and $T_\forall^*$ such that none of $\mathbb{A}^u$ from Phase1 satisfies any tree. Here $T_\forall^*$ is established such that any authorized user's attribute set could satisfy $T_\forall^*$, if and only if it satisfies any tree T_i, $i \in [1, N]$. The challenger first flips a random coin $\mu \in \{0, 1\}$, and chooses a random session key k, then computes the corresponding ciphertexts $\mathcal{E}_{T_\forall^*}(k)$, $CT_{m_\mu}^*$ and $CT_{\mathcal{N}_\mu}^*$. The ciphertexts are given to the adversary $\mathcal{A}$.

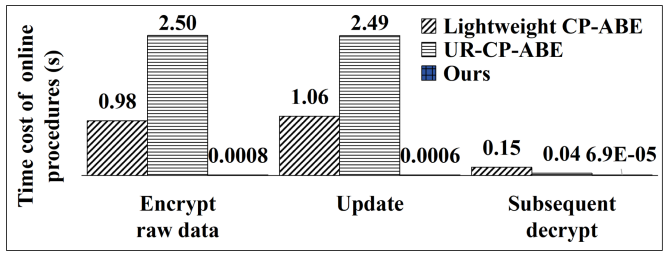


Fig. 9. Time comparison of key online procedures, including encrypt raw data, update, and subsequent decrypt at user.

Phase 2. Phase 1 is repeated under the condition that none of the sets of attributes $\mathbb{A}^{n_1+1}$, $\mathbb{A}^{n_1+2}$, …, $\mathbb{A}^{n_2}$ satisfies any access tree in Challenge.

Guess. The adversary $\mathcal{A}$ outputs a guess μ' of μ.

We define the advantage of the adversary $\mathcal{A}$ in this game as $Adv_\mathcal{A} = Pr[\mu' = \mu] - 1/2$.

Definition IV.2 (Scheme Security): We say that our FAPC scheme is secure if all probabilistic polynomial-time (PPT) adversaries $\mathcal{A}$ have at most a negligible advantage $Adv_\mathcal{A}$ in the above experiment.

The security of our FAPC scheme is proved using proof-by-reduction and game-based approaches. The proof logic is as follows. Let P_Π denote the problem of breaking the security of FAPC, P_Ω the problem of breaking the traditional CP-ABE

scheme, and $P_{\Omega'}$ the problem of breaking the traditional CP-ABE scheme with multiple encryptions (i.e., encrypting multiple messages under different access trees using the same public key). We first reduce $P_{\Omega'}$ to P_{Π} (Theorem 1), then prove the hardness of $P_{\Omega'}$ based on that of P_{Ω} (Theorem 2). Since P_{Ω} has been proven computationally hard [19], [23], the hardness of our problem P_{Π} follows from the reduction chain. Consequently, our FAPC scheme is secure under the security model defined in Section IV-B.

Theorem 1: If a PPT adversary has a non-negligible advantage in our security model (Section IV-B), then there exists another PPT adversary that can break the indistinguishable multiple encryptions of the traditional CP-ABE scheme [19] with a non-negligible advantage.

Theorem 2: If the traditional CP-ABE scheme [19] is CPA-secure, then it is also CPA-secure for CP-ABE scheme with multiple encryptions.

We refer readers to our previous work [1] to view the correctness and security proof details.

V. Identity-Differentiated Location Obfuscation

Using our cryptographic primitive FAPC [1] as the fundamental building block, we design a lightweight and secure sharing system. To fully demonstrate its practical utility, we present it in the context of location obfuscation, which is a widely studied application due to its importance in protecting user's geo-location privacy when using various LBS. We name our design *identity-differentiated location obfuscation* (IDLO). We will show that, enabled by our primitive FAPC, a geo-location DO can, through a semi-trusted helper, flexibly and efficiently control location privacy exposure towards an arbitrary number of data users. We will first introduce the model, then outline the policy enactment and version discovery modules, and finally detail the protocol design.

A. System Model

Our system consists of four major entities: DO, helper, data user, and a cloud-based data sharing platform, as shown in Fig. 2. The DO first encrypts the raw data d and uploads the ciphertext $E(d)$ to the data sharing platform. The helper manages the policy module, which mainly consists of a set of access trees $\mathcal{T}$ and a set of noises $\mathcal{N}$. The helper associates each noisy data $f(d, \epsilon_i)$ with one or more users' identities specified by the tree $T_i \in \mathcal{T}$. Note that enabled by our decoupling technique in FAPC, the policy module can run on a semi-trusted helper, thereby reducing the computational overhead on the DO. During runtime, each user fetches the encrypted data and, upon decryption, obtains their authorized version $f(d, \epsilon_i)$. Users may be either human individuals or third-party applications. As in ABE-based approaches, we also employ an attribute authority to generate decryption keys. The data d shared by the DO is disclosed to different users in different noisy versions with the assistance of helper.

B. Threat Model

We will consider the following three threat types:

1) *Curious users:* Users may attempt to derive other users' noisy versions. We also assume that collusion among users is possible, meaning multiple users might exchange information to jointly obtain data at a higher precision than what each is individually permitted to access.
2) *Honest-but-curious data sharing platform:* The data sharing platform honestly follows the protocol, but remains curious about all data stored on it.
3) *Semi-trusted helper:* The helper does not collude with any user, and is trusted to correctly enforce the precision policy, but it remains curious about the raw data. This mirrors the role of parents as reliable helpers for their children: They carefully set up access policies to protect their children's privacy, yet they may still want to pry into the children's private data.

C. Enacting Precision Sharing Policy

Location obfuscation has been extensively studied in multiple previous works [16], [17], [18]. The fine-grained precision sharing policy maps each user's identity to a specific obfuscated location. Following the approach in [24], we obfuscate locations based on the social closeness between each user and the DO: the closer the relationship, the higher the location precision authorized by the helper. Note that our framework supports arbitrary location-obfuscation schemes. This is orthogonal to our FAPC problem.

Noise Generation: We consider the real location as a small planar circular area with precise coordinate (x, y). The precise location area can be represented as (x, y, r_0), where r_0 is the area radius when optimal accuracy is achieved. For location obfuscation, the location noise, denoted as $(\Delta x, \Delta y)$, is chosen randomly and uniformly according to the location relevance parameter $\mathcal{R}$ [24]. This parameter is determined by the helper. It reflects the precision level of the data that a user can access. For each noisy version, the helper uses the corresponding $\mathcal{R}_i$ as input to derive the noise $(\Delta x_i, \Delta y_i)$. The noise generation process is described in Algorithm 1.

In Algorithm 1 the noise amplitudes are determined based on the relevance parameters (line 1). The higher value of $\mathcal{R}_i$ is, the smaller noise is chosen. The DO iteratively runs this algorithm N times, thereby generating N different noise values for N different precision levels.

D. Version Discovery

Our intent is to let the data sharing platform responsively locate the precision version stored on it when receiving requests from each user, such that the corresponding noise-component (identity-associated-component) ciphertexts are swiftly sent back to users. To this end, we leverage the bloom filter,[3] to

[3]In essence, bloom filter uses multiple hash functions to quickly check if an item is probably in a set, with the guarantee that it will never say an item is

Algorithm 1: Relevance-Based Noise Generation.

Input:
$\mathcal{R}_i \in (0, 1]$: location relevance parameter for the noisy version i;
(x, y, r_0): a planar circular area, where (x, y) is the precise location, and r_0 is the optimal radius;
Output:
$(\Delta x_i, \Delta y_i)$: the noise for the noisy version i;
1: compute the radius r_i as $r_i = r_0/\sqrt{\mathcal{R}_i}$;
2: randomly generate the noise $(\Delta x_i, \Delta y_i) \in \mathbb{Z}_q$ satisfying that the noisy location $(x - \Delta x_i, y - \Delta y_i)$ belongs to the annulus centered at (x, y) with inner radius r_0 and outer radius r_i;
3: **return** $(\Delta x_i, \Delta y_i)$

Algorithm 2: Index Matrix Generation.

Input:
$\mathcal{S}_i$: attribute set associated to the noisy version i;
Output:
IM_i: index matrix of the noisy version i;
1: denote the number of attributes in $\mathcal{S}_i$ as $l_i = |\mathcal{S}_i|$;
2: initialize IM_i as an $l_i \times K$ zero matrix;
3: **for** each attribute $\mathcal{S}_i[j]$, $j = 1$ to l_i, and $t = 1$ to K **do**
4: set $IM_i[j, t] = HASH(\mathcal{S}_i[j]|t)$;
5: **end For**
6: **return** IM_i

Algorithm 3: Filter Vector Generation.

Input:
$\mathbb{A}^u$: attribute set of the querying user u;
Output:
$\vec{v_u}$: u's filter vector;
1: initialize $\vec{v_u}$ as zero vector;
2: denote the number of attributes in $\mathcal{A}_u$ as $l_u = |\mathcal{A}_u|$;
3: **for** each attribute $\mathbb{A}^u[j]$, $j = 1$ to l_u, and $t = 1$ to K **do**
4: set $\vec{v_u}[HASH(\mathbb{A}^u[j]|t)] = 1$;
5: **end for**
6: **return** $\vec{v_u}$

Algorithm 4: Version Discovery.

Input:
$\vec{v_u}$: u's filter vector;
$\{IM_i\}_{i\in[1,N]}$: sets of index matrices of all noisy versions;
Output:
i or null;
1: outer: **for** each matrix IM_i, $i = 1$ to N **do**
2: denote l_i as the number of rows in IM_i;
3: **for** each $IM_i[j, t]$, $j = 1$ to l_i and $t = 1$ to K **do**
4: **if** $\vec{v_u}[IM_i[j, t]]! = 1$ **then** continue outer **end if**
5: **end for**
6: **return** i
7: **end for**
8: **return** null

locate the proper noisy version. The specific process consists of three steps:

Step 1 Index Matrix Generation for Noises: For each noisy version $i \in [1, N]$, the data sharing platform inputs the corresponding attribute set $\mathcal{S}_i$ into K hash functions. The resulting index matrix IM_i is used as vector indices, as outlined in Algorithm 2.

Step 2 Filter Vector Generation for Querying User: When a user u submits a data retrieval request along with an attribute set $\mathbb{A}^u$, the sharing platform feeds $\mathbb{A}^u$ to the K hash functions to obtain K indices. In the filter vector $\vec{v_u}$, all the bits at these K indices are set to 1, as shown in Algorithm 3. Fig. 3 gives an example of the filter vector generation for $\mathbb{A}^u$ when $K = 3$.

Step 3 Version Discovery: The platform runs Algorithm 4 to get the matching noisy version i. IM_i contains the hashing indices for version i's attribute set. If for all indices in IM_i the corresponding bits in $\vec{v_u}$ are 1, user u has the access permission and the algorithm returns i.

The major advantage of using Bloom filter is that the data sharing platform is definitely able to obtain the noisy version index, even when a user submits an incomplete attribute set. Moreover, the overhead of checking each noisy version remains constant, regardless of the type (e.g., integer, string) or number of attributes a user possesses. This is a significant benefit over other data structures for representing set, such as hash table. Note that this benefit comes at the cost of a small false-positive rate, meaning some noise ciphertexts may be mistakenly returned to the user. Nevertheless, protected by our cryptography primitive, such ciphertexts cannot be decrypted by the user if not permitted under the access policy.

Next, we present the IDLO protocol that utilizes our FAPC primitive as the fundamental building block, concatenated with the above noise generation and version discovery modules.

absent when it is actually present (no false negatives), though it may occasionally indicate presence incorrectly (allowing false positives).

E. IDLO Protocol Design

In the IDLO protocol, the crypto primitive's shared component represents data component, while the identity-associated components represent noise components. For simplicity, let d denote a raw location coordinate (x or y), and let ϵ_i denote the noise (Δx_i or Δy_i). We adopt an additive noise model, where a noisy data is defined as $d_{\epsilon_i} = f(d, \epsilon_i) = d - \epsilon_i$. Here, $i \in [1, N]$ is the noise index, and $d, \epsilon_i, d_{\epsilon_i} \in \mathbb{Z}_q$ represent the raw (i.e., noise-free) data, the noise, and the noisy data, respectively. Note that the subtraction operation is sufficient to realize all possible additive noise functions due to the modular arithmetic in $\mathbb{Z}_q$.

Our IDLO protocol consists of the following modules:

- **M1**—Setup: The attribute authority generates mk, pk by running FAPC.$Setup(1^\lambda)$, and generates $\mathbb{SK}_u$ for all users by running FAPC.$KeyGenerate(pk, mk, \mathbb{A}^u)$.
- **M2.1**—Access Policy Enacting (Helper-side): The helper first runs Algorithm 1 to obtain noises $\{(\Delta x_i, \Delta y_i)\}_{i\in[1,N]}$

for all users. Then the helper enacts the access control policy by establishing access trees $\mathcal{T} = \{T_i\}_{i\in[1,N]}$ and $T_\forall$.

- **M2.2**—Noise Space Conversion (Helper-side): Before encrypting, the helper needs to convert each noise $\epsilon_i \in \mathbb{Z}_q$ to $\tau_i = e(g,g)^{\epsilon_i} \in \mathbb{G}_T, i \in [1,N]$.
- **M2.3**—Noise Deployment (Helper-side): Given the inputs $\mathcal{N}_x = \{\tau_{x_i} = e(g,g)^{\Delta x_i}\}_{i\in[1,N]}$ and $\mathcal{N}_y = \{\tau_{y_i} = e(g,g)^{\Delta y_i}\}_{i\in[1,N]}$, the helper first chooses a symmetric session key k, then runs FAPC.$Encrypt''$ to derive the ciphertexts $CT_{\mathcal{N}_x}$, $CT_{\mathcal{N}_y}$, and $\mathcal{E}_{T_\forall}(k)$. All ciphertexts are uploaded to the sharing platform, while the symmetric key k is secretly sent to the DO. Additionally, the helper runs Algorithm 2 and sends the index matrices $\{IM_i\}_{i\in[1,N]}$ along with the ciphertexts.
- **M3.1**—Precise Location Space Conversion (DO-side): Before encrypting, the DO needs to convert the raw location $d \in \mathbb{Z}_q$ to $m = e(g,g)^d \in \mathbb{G}_T$.
  Correspondingly, the noisy data is converted as $m_{\tau_i} = e(g,g)^{d_{\epsilon_i}}$, and the noisy function is converted to the multiplicative form as $f'(m,\tau_i) = m_{\tau_i} = \frac{m}{\tau_i}$. Note that when $\epsilon_i = 0$, we have $f(d,\epsilon_i) = d$ and $f'(m,\tau_i) = m$.
- **M3.2**—Data Publishing (DO-side): When first publishing data and given the inputs $m_x = e(g,g)^x$ and $m_y = e(g,g)^y$, the DO runs FAPC.$Encrypt'$ to derive the ciphertexts CT_{m_x} and CT_{m_y}. In the update phase, given the new raw data $m'_x = e(g,g)^{x'}$, $m'_y = e(g,g)^{y'}$ as input, the DO runs FAPC.$Update$.Step 1 and outputs $CT_{m_x\to m'_x}$ and $CT_{m_y\to m'_y}$. All ciphertexts are uploaded to the data sharing platform.
- **M3.3**—Data Publishing (Cloud-side): Given $CT_{m_x\to m'_x}$ and $CT_{m_y\to m'_y}$, the cloud runs FAPC.$Update$.Step 2 to derive the ciphertext $CT_{m'_x}$ and $CT_{m'_y}$ for new raw data m'_x and m'_y.
- **M4.1**—Version Discovery (Cloud-side): Upon receiving a user u's query request, the data sharing platform first applies the Bloom filter as presented in Algorithm 3 to generate a filter vector, then checks each noisy version by running Algorithm 4 to obtain the matching noisy version i, finally pushes the corresponding noise-ciphertext component $\{\mathcal{E}_{T_i}(A_{\tau_{x_i}}), \mathcal{E}_{T_i}(A_{\tau_{y_i}})\}$ and the session key ciphertext $\mathcal{E}_{T_\forall}(k)$ to user u.
- **M4.2**—Noise Partial-decryption (User-side): After receiving $\{\mathcal{E}_{T_i}(A_{\tau_{x_i}}), \mathcal{E}_{T_i}(A_{\tau_{y_i}})\}$ and $\mathcal{E}_{T_\forall}(k)$, the user u runs FAPC.$Decrypt$.Steps 1-2 to obtain the plain session key k and the noise-hidden components $\{A_{\tau_{x_i}}, A_{\tau_{y_i}}\}$.
- **M4.3**—Data Reveal (User-side): Each user first downloads $CT_{m'_x}, CT_{m'_y}$, then runs FAPC.$Decrypt$.Step 3 to obtain $f'(m'_x,\tau_{x_i}) = \frac{m'_x}{\tau_{x_i}}$, $f'(m'_y,\tau_{y_i}) = \frac{m'_y}{\tau_{y_i}}$ or null.
- **M4.4**—Inverse Space Conversion (User-side): The users convert $f'(m'_x,\tau_{x_i})$ and $f'(m'_y,\tau_{y_i})$ back to the space $\mathbb{Z}_q$ by using Pollard's Kangaroo (a.k.a, Pollard's Lambda) algorithm [25] or looking up the table containing all the possible noisy data [26], [27]. As we know, efficiently solving the discrete logarithm problem requires small data space. Assuming the location precision unit is 1 meter and the message space is always less than 20 bits, then the supported location range is $2^{20}\times 2^{20}m^2$, which is sufficiently large for the location sharing applications. By following this step, users could obtain the newest updated noisy data $(x' - \Delta x_i, y' - \Delta y_i)$.

The overall protocol is shown in Fig. 4. We divide it into three parts. 1) In the initialization part, the attribute authority runs **M1**. Each user fetches its secret attribute key from attribute authority. Meanwhile the helper runs **M2.1-M2.3** to set, convert and encrypt the noises, finnally uploads the noises' ciphertexts to the cloud. 2) In the online data publishing part, the DO runs **M3.1-M3.2** to encrypt the realtime location and uploads the ciphertexts to the cloud. The cloud runs **M3.3** to update the ciphertexts. 3) In the data retrieval part, the cloud runs **M4.1** upon receiving query request from users. The noise ciphertexts are then sent to the users. The users runs **M4.2** to perform pre-decryption. In runtime, each user downloads the latest data-component ciphertexts and runs **M4.3** and **M4.4** to reveal the authorized-obfuscated version of location data.

VI. Complexity Analysis

In our scheme, the $Setup$ algorithm computes a primary key mk and a system public key pk, with computational complexity $O(1)$. The $Encrypt'$ algorithm incurs constant overhead when encrypting C_m and P_{up}. In $Encrypt''$ algorithm, the helper chooses a polynomial p_x for each node x in every tree $T_{i,i\in[1,N]}$ and in $T_\forall$. This step dominates the overhead, with complexity $O(N\cdot X)$, where N is number of noise levels and X is the number of leaves per access tree. This overhead is incurred only once during real-time data sharing. When one noise must be updated (e.g., changing precision level), the helper simply invokes $Encrypt''$ on new noise with complexity $O(X)$. While in $Update$ algorithm, it mainly refers to the common update of raw data. The DO only needs to compute $\{\Delta C_{m'}, E_k(P'_{up})\}$, and send them to the cloud. The cloud then performs a multiplication to generate $C_{m'}$. Thus, the update complexity on both the DO and cloud sides, as well as the communication overhead, are $O(1)$. The $KeyGenerate$ algorithm computes an attribute key from each user attribute in $\mathbb{A}^u$. Its computational complexity is $O(|\mathbb{A}^u|)$. As for $Decrypt$ algorithm, the first decryption incurs complexity $O(X)$. This is because it requires two calls to CP-ABE.$Decrypt$ algorithm (one to decrypt the session key k and another to decrypt the noise-hidden component A_{τ_i}), followed by a simple symmetric decryption, one multiplication and one division to recover the final noisy data. Both k and A_{τ_i} can be reused in subsequent decryptions, reducing the complexity of each subsequent decryption to $O(1)$.

We compare FAPC with two representative CP-ABE schemes: lightweight CP-ABE [20] and UR-CP-ABE [21]. Both are recent advances in ABE. Lightweight CP-ABE eliminates bilinear pairings for IoT environments. UR-CP-ABE offers flexible access control with binary-tree optimization. For a fair comparison, we disable UR-CP-ABE's user revocation feature (i.e., keeping the revocation list empty), thereby eliminating its associated overhead. This allows us to evaluate core encryption

and decryption performance under identical conditions. Table II summarizes computational complexities.

The comparison results highlight the efficiency advantages of our scheme. For $Setup$, ours achieves constant complexity $O(1)$, whereas lightweight CP-ABE scales linearly with attribute space size $|AS|$ and UR-CP-ABE additionally depends on the maximum number of users $|NU|$. In $KeyGenerate$, both lightweight CP-ABE and our scheme are linear in the user's attribute count $|\mathbb{A}^u|$, while UR-CP-ABE incurs an extra logarithmic factor $\log|NU|$ due to its binary tree. The most notable gains are in the online encryption and update. Our scheme offloads heavy work to a helper (Encrypt at Helper: $O(N \cdot X)$), so the DO performs only constant-time operations for encryption ($O(1)$) and update ($O(1)$). In contrast, lightweight CP-ABE and UR-CP-ABE require the DO to carry out $O(N \cdot X)$ work for both encryption and update, which becomes costly when N or X is large. For $Decrypt$, our scheme achieves $O(X)$ for the first decryption and $O(1)$ for subsequent ones, matching the asymptotic efficiency of the other two schemes after the initial overhead. Overall, our design is the most suitable for scenarios requiring frequent data updates and low-latency online operations at the DO side.

VII. Implementation

A. Implementation

Based on our protocol design, we implement a location-obfuscation system, which contains a cloud-based platform for open-access data retrieving, and client-side applications. All algorithms in our FAPC scheme are implemented using the Java Pairing-based Cryptography (JPBC) library [28]. Our system is deployed on Amazon EC2 t2.large machine (as the cloud) with 8 GB of RAM, and multiple desktops as the DO, helper, user, and attribute authority with the 13th Gen Intel(R) Core(TM) i7-1360P (2.20 GHz) and 32.0G RAM. The back-end of our cloud-based platform is written in PHP, performing the ciphertext storage & update at the cloud. The front-end applications run at the client side, performing privacy-preserving location data publishing, retrieving and visualization. The location-data ciphertexts are stored in MySQL database and retrievable by third-party user-applications through $http$ connections. The architecture of our system is shown in Fig. 5.

B. A Case Study

We use a practical example to demonstrate our system. We select the coordinate (longitude: 116.397026, latitude: 39.918058) as the DO's raw location. Let $r_0 = 10$ meters be the area radius with the optimal accuracy, i.e., any published location within this range is considered precise. We demonstrate our system with a 32-user scenario. Assume the helper defines three location relevance parameters: $\mathcal{R}_0 = 1.00, \mathcal{R}_1 = 0.04, \mathcal{R}_2 = 0.01$. The corresponding radii are $r_0 = 10, r_1 = 50, r_2 = 100$. Thus there are 3 precision levels. Fig. 6 shows these 32 users: User 0 and User 1 belong to the highest precision level, User 2 to User 11 belong to the medium precision level, and all others belong to the lowest precision level. The protocol is executed as described in Section V-E. All uploaded ciphertexts are stored in a MySQL database.

Each user downloads and decrypts the ciphertext with the attribute key issued by the attribute authority. The user will obtain an obfuscated version if his attribute set satisfies the corresponding access tree. Due to space limitation, we only show the decrypted locations for User 2 and User 31 in Fig. 7. We can see that User 2 and User 31 access the published location with different precisions (50 m, 100 m), while exactly matching the precisions assigned by the helper (Fig. 6). More importantly, the DO could efficiently update its location by only changing the data component ciphertext, while ensuring different obfuscated-location sequences observed by different users in runtime.

C. Performance Analysis

We conduct experiments to evaluate both the offline algorithms ($Setup$, $KeyGenerate$, $Encrypt''$) and online algorithms ($Encrypt'$, $Update$, $Decrypt$). To assess the feasibility in real-time location sharing, we compare our scheme with two state-of-the-art CP-ABE schemes, i.e., lightweight CP-ABE [20] and UR-CP-ABE [21]. As discussed in Section III-B, these two schemes can be adapted to solve the IDLO problem, and their computational complexities were compared in Section VI.

Setup. As shown in Fig. 8(a), our scheme has constant time overhead of computing mk and pk. The average running time over 100 independent tests is 13.2 ms. In contrast, lightweight CP-ABE and UR-CP-ABE require predefining the attribute space and performing computations for each attribute. Thus, their setup cost scales linearly with the attribute space size.

Key Generate. The computational complexity is $O(W)$, where W is the number of one user's attributes. To validate this, we randomly select the user attributes with counts ranging from 10 to 100. As shown in Fig. 8(b), lightweight CP-ABE achieves the shortest key generation time and the slowest growth with increasing attributes, UR-CP-ABE exhibits moderate linear growth, while our scheme has the longest runtime and the fastest growth rate. Fortunately, key generation is one-time and offline per user, making this weakness acceptable.

Encrypt. We first compare the total encryption time of our scheme with lightweight CP-ABE and UR-CP-ABE under a 16-leaf tree, as shown in Fig. 8(c). Our scheme achieves moderate encryption time that scales smoothly with the number of noise levels N, significantly outperforming UR-CP-ABE. Although slightly slower than lightweight CP-ABE, it strikes a favorable balance between functionality and efficiency.

We then measured the detailed encryption time of our scheme for N=1 to 10 under both 16-leaf and 32-leaf access trees, as shown in Fig. 8(d) and (e). The lines marked "$\times$" and "$+$" represent the time of encrypting the session key k and the noises, respectively. Both are directly proportional to the number of leaves X per access tree, and the noise encryption time is also directly proportional to N. Fortunately, these operations are one-time, offline, and handled by the helper. Fig. 8(d) and (e) also show the online algorithm $Encrypt'$ (marked "$\triangle$ "),

which is constant and low (around 0.8 ms). This is because $Encrypt'$ only encrypts the raw data m, involving two exponentiation, one multiplication and one symmetric encryption. Consequently, when real-time location is updated, only a very small computational cost is required to update the ciphertext. Next, we present $Update$ performance.

Update. Theoretically, both communication and computational complexities of data update are constant and independent of the number of noises, thanks to the decoupled encryption of the raw data (at the DO) and noise (at the helper). We validate this with experiments using different numbers of noise levels. As shown in Fig. 8(f) and (g), our scheme exhibits exceptional update performance. Both execution time and communication cost remains consistently minimal (around 0.6 ms and 284 bytes) and are independent of the noise level counts. In contrast, lightweight CP-ABE and UR-CP-ABE show update overhead that grows significantly with noise count, highlighting the clear advantage of our design in dynamic scenarios. Because update communication and computation cost are constant, our scheme scales to support a large number of users simultaneously.

Decrypt. Fig. 8(h) shows the decryption time over multiple updates. Lightweight CP-ABE maintains a stable decryption time around 0.15 s, and UR-CP-ABE around 0.04 s. Our scheme incurs a slightly higher overhead of 0.20 s for the first decryption, as it requires three steps: decrypting the session key k, decrypting the noise-hidden component A_{τ_i}, and then computing the composite of shared and identity-associate components. However, for all subsequent decryptions, the overhead drops dramatically to around 0.000069 s, because k and A_{τ_i} can be reused.

Fig. 9 shows the time cost of three key online procedures (including encrypt raw data, update, and subsequent decrypt), demonstrating our scheme's substantial improvements over lightweight CP-ABE and UR-CP-ABE.

In real-time location sharing, our scheme achieves $O(1)$ online encryption and update overhead ($<$ 1ms latency and 284-byte communication), and reduces subsequent decryptions to $O(1)$ after the first $O(X)$ decryption, enabling efficient handling of frequent location updates.

VIII. Discussion

Complexity. Our encryption scheme relies on pairing operations. According to our experimental results, encrypting each data component (shared component) takes around 0.8 ms, and each decryption takes around 0.069 ms. This overhead is negligible for the DO and users, as it is independent of the number of noise levels and the complexity of access policies. In contrast, a direct CP-ABE-based solution would incur significant computational and communication costs for encrypting noise components and decrypting composite data, and these costs would be repeated with every data update.

Security. Some applications may require dynamic changes to noise components for higher security. In such cases, each session should be kept short. For example, a real-time geo-location data sharer may want to periodically randomize the obfuscating noise to defend against side-channel attacks that exploit road-layout information (e.g., by fitting noisy location traces to a city map). Under our framework, this can be handled by pre-computing all noise components and uploading them at runtime. Because the data and noise components are decoupled, they can be processed in parallel. The DO can therefore delegate noise-component generation, update and policy enactment to a helper (an entity he/she trusts), while focusing solely on the lightweight task of protecting the raw data. Issues such as revoking obsolete noise components can be resolved by simply updating the random numbers $\hat{s}$, $\tilde{s}$.

Application. From an application perspective, our generic primitive can be useful for protecting patients' sensitive medical data in the public medical data centers, which collect personal health information from nation-wide hospitals and institutions [29], [30]. Adding different noise levels has been extensively studied for queries' privacy preservation in differential privacy [31]. However, it remains unclear how to flexibly and efficiently publish data with varying privacy leakage levels for different data users or auditors, largely due to technical challenges. Fine-grained control over video/image resolution is also valuable for enabling identity-differentiated streaming quality in multimedia content-sharing services [32], [33]. We believe that applying our FAPC technique will pave the way for achieving more secure and lightweight fine-grained control across all these domains.

IX. Related Work

Fine-grained Access Control. Cryptography-based fine-grained access control[4] allows a DO to flexibly bind each piece of data with specific user identities for data usage. With this approach, DO does not need to trust any centralized gateway, as used in the traditional access control, which has been reported for abusing data without permissions [34].

As an advancement, ABE defines a user identity by his/her attribute set. Sahai and Waters first proposed ABE [35] to enforce access control over encrypted data. Goyal et al. later extended it to key-policy ABE (KP-ABE) [36] and Bethencourt et al. to CP-ABE [19]. In KP-ABE, a user's secret key is associated with an access policy over attributes. The user can decrypt if and only if the ciphertext's attribute set satisfies the access policy. In CP-ABE, the ciphertext is associated with an access policy, and the user's private key is tied to an attribute set. Decryption succeeds if and only if the attribute set satisfies the policy. Thus, DO can decide who can access the data. Multiple ABE approaches have been proposed for secure data outsourcing [20], sharing [21], and keyword searching [11], [37], [38].

However, the practical use of fine-grained access control is limited because it lacks support for expressive function-level control: access control targets only the data itself, not its functional operations. Directly applying existing ABE schemes to our access-precision problem incurs significant overhead for the DO (see Section III-B). Adding functional features to fine-grained access control remains an open problem, which requires specifying not only which user could access what data, but also how to use that data. Enhancing access control from data

[4]For simplicity, we denote it as fine-grained access control in this paper.

level to function level is critical for realizing a more expressive mechanism under the DO's authority.

Recently several researchers have started combining ABE with HE [8], [9], [10], [11], [12], [13], [14], [15], enabling arithmetic operations over ciphertexts associated with different attributes. However, none of these works consider homomorphic operation between a shared component and identity-associated components, where only the composite of the two is accessible to an authorized user. The enlarged-ciphertext-size problem also remains open in most of these works. Moreover, in our scheme, all communication between parties is minimal and does not rely on interactive collaboration among multiple parties.

Homomorphic Encryptions. In our scheme, the combination of the shared component and identity-associated components relies on homomorphic properties. Homomorphic encryption has been extensively studied in privacy-preserving computing. Partially or fully homomorphic encryption allows arithmetic operations to be performed on ciphertexts in the cloud, revealing only the final operation results [39]. Leveraging these cryptographic primitives, various utility functions can be evaluated on untrusted nodes, enabling applications such as range search [40], [41], keyword search [42], genomic data search [43], image search [44], and clustering [45], etc.

However, none of these approaches support fine-grained access control, i.e., they do not differentiate data or its usage across different user identities. Moreover, all these approaches focus on functional operations and lack adequate support for fine-grained control in a multi-user model.

X. Conclusion

In this work, we present a privacy-preserving data precision tuning scheme over outsourced data through a semi-trusted helper. This scheme targets non-technical data owners who wish to tune data precision and share with different users, but are unable to perform these tasks independently. To this end, we delegate a helper to carry out data precision tuning and access policy enforcement. Since the helper is considered semi-trusted, we utilize our crypto primitive FAPC to split the noisy data ciphertext into a dynamically shared component (raw data ciphertext) and an identity-associated component (noise ciphertext). In this way, the privacy of raw data is protected from all parties, including the helper. This approach overcomes the challenge of delegating control over data precision while preserving data privacy. Finally, we implement our system on a commercial cloud platform and conduct extensive experiments to validate its practical performance.

Acknowledgments

Part of this work was published in AsiaCCS 2019 [1].

References

[1] Q. Xie, Y. Hou, K. Cheng, G. G. Dagher, L. Wang, and S. Yu, "Flexibly and securely shape your data disclosed to others," in *Proc. ACM Asia Conf. Comput. Commun. Secur.*, 2019, pp. 160–167.

[2] Y. Yang, J. Li, and X. Zhang, "Privacy-preserving against gradients leakage attacks via joint differential privacy in federated learning," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 10, no. 3, pp. 2456–2470, Jun. 2026.

[3] Y. Huang, T. Ni, Y. Liu, P. Hu, Q. Yu, and Y. Luo, "Location privacy protection method based on local differential privacy in crowdsensing with approximately accurate task allocation," *IEEE Trans. Services Comput.*, vol. 19, no. 1, pp. 463–476, Jan./Feb. 2026.

[4] Y. Zhao, K. Zhang, L. Gao, and J. Chen, "Privacy and fairness analysis in the post-processed differential privacy framework," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 2412–2423, 2025.

[5] M. Liberato, A. Affinito, B. Meijerink, M. Jonker, A. Botta, and A. Sperotto, "To block or not to block? Evaluating parental controls across routers, DNS services, and software," in *Proc. 9th Netw. Traffic Meas. Anal. Conf.*, 2025, pp. 1–10.

[6] A. S. Rafsanjani, S. Aslam, M. SaberiKamarposhti, M. B. Jasser, M. Tahir, and S. L. Lau, "KeyAware: An ethical parental monitoring keylogger for child online safety," in *Proc. IEEE 13th Conf. Syst. Process Control*, 2025, pp. 187–192.

[7] M. B. M. Stoilova and S. Livingstone, "Do parental control tools fulfil family expectations for child protection? A rapid evidence review of the contexts and outcomes of use," *J. Child. Media*, vol. 18, no. 1, pp. 29–49, 2024.

[8] L. Yang, Y. Yang, D. Niyato, Z. Li, W. Xia, and L. Sun, "Dynamic searchable symmetric encryption with efficient and complete access control for multi-user cloud computing," *IEEE Trans. Mobile Comput.*, vol. 25, no. 2, pp. 2825–2842, Feb. 2026.

[9] L.-Y. Yeh, S.-P. Tseng, C.-H. Lu, and C.-Y. Shen, "Auditable homomorphic-based decentralized collaborative AI with attribute-based differential privacy," *IEEE Trans. Netw. Service Manag.*, vol. 22, no. 2, pp. 989–1004, Apr. 2025.

[10] K. Emura, S. Sato, and A. Takayasu, "Attribute-based keyed fully homomorphic encryption," in *Proc. Int. Conf. Secur. Cryptography Netw.*, 2024, pp. 47–67.

[11] M. Wang, Y. Miao, Y. Guo, H. Huang, C. Wang, and X. Jia, "AESM^2 attribute-based encrypted search for multi-owner and multi-user distributed systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 1, pp. 92–107, Jan. 2023.

[12] W. Ding, Z. Yan, X. Qian, and R. H. Deng, "Computing maximum and minimum with privacy preservation and flexible access control," in *Proc. IEEE Glob. Commun. Conf.*, 2019, pp. 1–7.

[13] W. Ding, Z. Yan, and R. H. Deng, "Privacy-preserving data processing with flexible access control," *IEEE Trans. Dependable Secure Comput.*, vol. 17, no. 2, pp. 363–376, Mar./Apr. 2020.

[14] Z. Brakerski, D. Cash, R. Tsabary, and H. Wee, "Targeted homomorphic attribute-based encryption," in *Proc. Theory Cryptogr. Conf.*, 2016, pp. 330–360.

[15] C. Gentry, A. Sahai, and B. Waters, "Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based," in *Proc. Annu. Cryptol. Conf.*, 2013, pp. 75–92.

[16] S. Hong and L. Duan, "Location privacy protection game against adversary through multi-user cooperative obfuscation," *IEEE Trans. Mobile Comput.*, vol. 23, no. 3, pp. 2066–2077, Mar. 2024.

[17] B. Ma et al., "A joint trajectory obfuscation and pseudonym swapping mechanism avoiding extra privacy cost," *IEEE Trans. Intell. Transp. Syst.*, vol. 27, no. 6, pp. 6994–7010, Jun. 2026.

[18] C. Xu, Y. Ding, C. Chen, Y. Ding, W. Zhou, and S. Wen, "Personalized location privacy protection for location-based services in vehicular networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 1, pp. 1163–1177, Jan. 2023.

[19] J. Bethencourt, A. Sahai, and B. Waters, "Ciphertext-policy attribute-based encryption," in *Proc. IEEE Symp. Secur. Privacy*, 2007, pp. 321–334.

[20] X. Yao, J. Zhou, X. Du, and S. Zhang, "A CP-ABE and IOTA-based lightweight sensitive data access control scheme for IoT," *IEEE Internet Things J.*, vol. 11, no. 24, pp. 40831–40844, Dec. 2024.

[21] Z. Guo et al., "UR-CP-ABE: CP-ABE with flexible construction mechanism and efficient user revocation capability for access control in the cloud," *IEEE Trans. Dependable Secure Comput.*, vol. 23, no. 3, pp. 5989–6004, May/Jun. 2026.

[22] J. Krumm, "A survey of computational location privacy," *Pers. Ubiquitous Comput.*, vol. 13, no. 6, pp. 391–399, 2009.

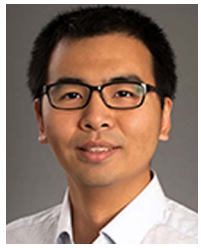
[23] Z. Wan, J. Liu, and R. H. Deng, "HASBE: A hierarchical attribute-based solution for flexible and scalable access control in cloud computing," *IEEE Trans. Inf. Forensics Security*, vol. 7, no. 2, pp. 743–754, Apr. 2012.

[24] C. A. Ardagna, M. Cremonini, S. D. C. di Vimercati, and P. Samarati, "An obfuscation-based approach for protecting location privacy," *IEEE Trans. Dependable Secure Comput.*, vol. 8, no. 1, pp. 13–27, Jan./Feb. 2011.

[25] J. M. Pollard, "Monte Carlo methods for index computation (mod p)," *Math. Computation*, vol. 32, no. 143, pp. 918–924, 1978.

[26] E. Shi, H. Chan, E. Rieffel, R. Chow, and D. Song, "Privacy-preserving aggregation of time-series data," in *Proc. Annu. Netw. Distrib. Syst. Secur. Symp.*, 2011.
[27] B. Wang, M. Li, S. S. Chow, and H. Li, "Computing encrypted cloud data efficiently under multiple keys," in *Proc. IEEE Conf. Commun. Netw. Secur.*, 2013, pp. 504–513.
[28] A. De Caro and V. Iovino, "JPBC: Java pairing based cryptography," in *Proc. IEEE Symp. Comput. Commun.*, 2011, pp. 850–855.
[29] F. A. Alijoyo, M. B. Alazzam, A. H. H. Muhammad, A. H. Mutlag, M. Z. N. Al-Dabagh, and B. K. Bala, "Ethical implications of data sharing and patient privacy in medical informatics systems," in *Proc. Int. Conf. Elect. Comput. Energy Technol.*, 2025, pp. 1–5.
[30] Q. Zhang, X. Xue, and J. Yang, "Blockchain-enabled trustworthy healthcare data sharing mechanism for reliable 6G-IoT networks," *IEEE Internet Things J.*, vol. 13, no. 5, pp. 7738–7748, Mar. 2026.
[31] N. Küchler, A. Viand, H. Lycklama, and A. Hithnawi, "DPolicy: Managing privacy risks across multiple releases with differential privacy," in *Proc. IEEE Symp. Secur. Privacy*, 2025, pp. 3950–3968.
[32] R. Zhao, Y. Zhang, J. Ji, S. Yi, W. Wen, and R. Lan, "AES-AUDIO: An encryption scheme for audio supporting differentiated decryption," *IEEE Trans. Multimedia*, vol. 27, pp. 2268–2280, 2025.
[33] K. B. A. Kumar, L. S. Mohith, K. Jain, P. Krishnan, N. Venkatachalam, and R. Buyya, "Post-quantum cryptography-based multimedia encryption communication scheme in IoT consumer electronics," *IEEE Trans. Consum. Electron.*, vol. 71, no. 2, pp. 4995–5006, May 2025.
[34] C. Cadwalladr and E. Graham-Harrison, "Revealed: 50 million Facebook profiles harvested for Cambridge analytica in major data breach," *Guardian*, vol. 17, 2018, Art. no. 22.
[35] A. Sahai and B. Waters, "Fuzzy identity-based encryption," in *Proc. Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, 2005, pp. 457–473.
[36] V. Goyal, O. Pandey, A. Sahai, and B. Waters, "Attribute-based encryption for fine-grained access control of encrypted data," in *Proc. 13th ACM Conf. Comput. Commun. Secur.*, 2006, pp. 89–98.
[37] S. Prasad and Y. S. Rao, "Fine-grained authorized encrypted-data retrieval mechanism based on Boolean query searchable KP-ABE framework," *IEEE Internet Things J.*, vol. 13, no. 2, pp. 1778–1797, Jan. 2026.
[38] H. Wang, J. Ning, X. Huang, G. Wei, G. S. Poh, and X. Liu, "Secure fine-grained encrypted keyword search for E-healthcare cloud," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 3, pp. 1307–1319, May/Jun. 2021.
[39] C. Gentry, "A fully homomorphic encryption scheme," PhD dissertation, Stanford University, 2009.
[40] H. Wu, Z. Peng, J. Xiao, L. Xue, C. Lin, and S.-H. Chung, "HeX: Encrypted rich queries with forward and backward privacy using trusted hardware," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 4, pp. 3751–3765, Jul./Aug. 2025.
[41] F. Liu et al., "Volume-hiding range searchable symmetric encryption for large-scale datasets," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 4, pp. 3597–3609, Jul./Aug. 2024.
[42] N. Wang, W. Zhou, J. Wang, J. Fu, J. Liu, and B. K. Bhargava, "VLMS: Verifiable lattice-based encryption with multi-keyword search in cloud storage," *IEEE Trans. Dependable Secure Comput.*, to be published, doi: 10.1109/TDSC.2026.3675085.
[43] S. Jafarbeiki, A. Sakzad, S. K. Kermanshahi, R. Steinfeld, and R. Gaire, "PRESSGenDB: Privacy-preserving substring search on encrypted genomic database," in *Proc. IEEE Conf. Comput. Commun. Workshops*, 2022, pp. 1–6.
[44] M. S. Uzzal, I. Ahmad, and S. Shin, "SCBIR-PE: Secure content-based image retrieval with perceptual encryption," *IEEE Trans. Dependable Secure Comput.*, vol. 23, no. 2, pp. 1940–1954, Mar./Apr. 2026.
[45] Y. Chang, Y. Li, T. H. Luan, Y. Wang, J. Zheng, and Z. Su, "Privacy-preserving K-means clustering for vehicular driving behavior analysis," *IEEE Trans. Netw. Sci. Eng.*, vol. 13, pp. 4930–4945, 2026.

Qingqing Xie received the PhD degree in computer application technology from Anhui University, China, in 2017. She is an associate professor with the College of Computer Science and Communication Engineering, Jiangsu University, China. From September 2016 to September 2017, she visited Boise State University, Department of Computer Science, as a visiting scholar. Her current research interests include blockchain, cloud computing, and applied cryptography.



Yantian Hou (Member, IEEE) received the PhD degree from Computer Science Department, Utah State University, in 2016. He is a researcher with Palo Alto Networks. He was a visiting research assistant with the University of Arizona from 2015 to 2016, and an assistant professor with Boise State University from 2016 to 2023. His research interests include AI agent security, wireless network security, and cloud computing security.

Fatong Zhu received the MS degree from Jiangsu University, Zhenjiang, China, in 2025. His research interest include applied cryptography.

Ke Cheng received the PhD degree in computer science and technology from Xidian University, Xi'an, Shaanxi, China, in 2022. He is an associate professor with the School of Computer Science and Technology, Xidian University. His research interests include data security and privacy protection.

Hanyu Quan (Senior Member, IEEE) received the MS and PhD degrees in information security and cryptography from Xidian University, in 2014 and 2019, respectively. He is a lecturer with the College of Computer Science and Technology, Huaqiao University, Xiamen, China. From 2015 to 2017, he was a visiting PhD student with the Wireless Network and Cyber Security Research Laboratory, Department of Electrical and Computer Engineering, University of Arizona, USA. His research interests include privacy-enhancing technologies and applied cryptography.